Sorting - 3

Agenda:

1. Partition Array (TC = O(N)) (Return index of partition point)
2. Quick Sort
3. Time Complexity analysis of Quick Sort
4. Inversion Count

Partion Index

Q. Given an array A of length N, considering the last element as pivot p, rearrange the elements such that for all i:

T.C = O(N)

- if A[i] < p then it should be present on left side of the partition.
- If A[i] > p then it should be present on right side of the partition

⇒ return the partition index.

Sol.

```
class Solution {
  public int Partition (int[] arr) {
    int n = arr.length;
    int pivot = arr[n-1];      → Pivot is choosen as the last element
    int i = 0;                 → Pointer track where next smaller element should go
    int j = n-2;               → Pointer scans from the right side.

    while (i <= j) {           // Loop until both the pointers cross
        if (arr[i] <= pivot) {     // If current element at i is ≤ to pivot,
            i++;                   if already or left side move i forward.
        } else if (arr[j] > pivot) {   // If current element at j is > than pivot,
            j--;                       it belong to the right side, so j move
        } else {                       backward.
            int temp = arr[i];
            arr[i] = arr[j];           // otherwise swap them.
            arr[j] = temp;
        }
    }
    int temp = arr[n-1];       (At this point all elements
    arr[n-1] = arr[i];          before i are <= Pivot
    arr[i] = temp;              and after i are > Pivot
                                So, swap the pivot with i
    return i;
  }
}
```

Quick Sort (Divide & Conquer)

ex: A = [54, 26, 93, 17, 77, 31, 44, 55, 20] TC = O(n)

0 1 2 3 4 5 6 7 8

↳ Pivot

[26, 17, 31, 44, 20] 54 [93, 77, 55]

[17, 20] 26 [31, 44] [77, 55] 93

Partition index

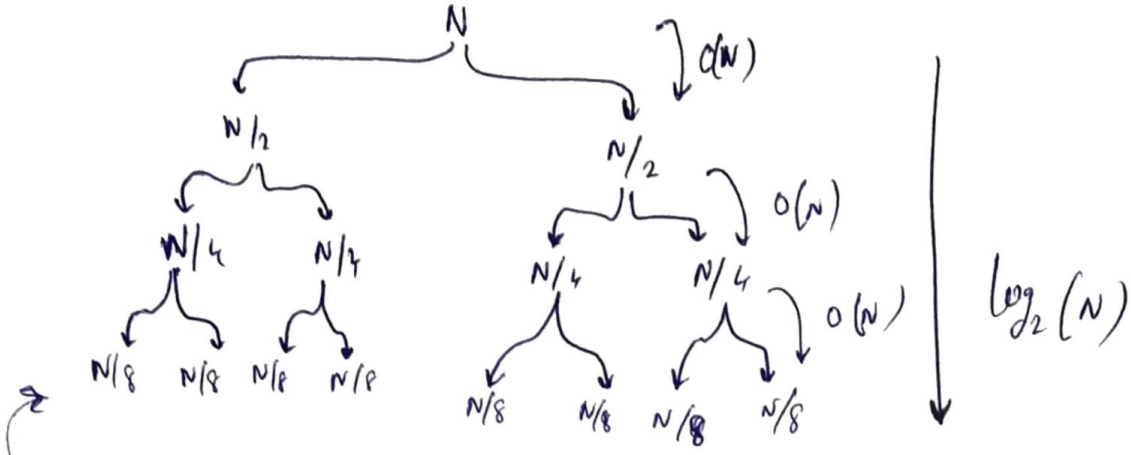
17 [20] 31 [44] [55] 77

↳ [17, 20, 26, 31, 44, 55, 77, 99

Sol

```
public class Solution{
    public int[] solve (int[] A){
        quickSort (A, 0, A.length-1);
        return A}
    }

    private void quickSort (int[] arr, int low, int high){
        if (low < high){
            int pivotIndex = partition (arr, low, high);
            quickSort (arr, low, pivotIndex -1);
            quickSort (arr, pivotIndex +1, high);
        }
    }

    private int partition (int [] arr, int low, int high){
        int pivot = arr[high];
        int i = low;
        int j = high - 1;

        while (i<=j){
            if (arr[i] <= pivot){
                i++;
            }
            else if (arr[j] > pivot){
                j--;
            }
            else {
                swap (arr, i, j);
            }
        swap(arr, i, high);
        return i
    }

}
```

T.C



Best Case → $TC = \Theta(N \log N)$
$SC = O(\log N)$

Worst Case → $TC = O(N^2)$
$SC = O(N)$

[38, 9, 17, 20, 30]
N-1
[8, 9, 10, 20, 30]
N-2
[9, 10, 20, 30]
N-3

Inversion Count → count of pair of indices (i, j), where i<j but A[i] > A[j].

- Merge Sort Approach (N log N T.C)

DSA Sorting-3 (Q.2)

```
public class Solution {
    Public int solve(int[] A)
        long modulo = 1000000007L;
        long result = countInversions(A, 0, A.length -1) % modulo
        return (int) result;
    }

    Private long countInversions(int[] arr, int l, int r){
        long count = 0;
        if (l<r) {
            int m = (l+r)/2;

count += countInversions(arr, l, m); // count inversions in left half
count += countInversions(arr, m+1, r); // count inversions in right half
count += mergeAndCount(arr, l, m, r); // count cross inversions while merging
}
return count;
}
```

```
//merge 2 sorted halves and count inversions.
private long mergeAndCount (int[] arr, int l, int m, int r){
    int [] left = Arrays.copyOfRange (arr, l, m+1);
    int [] right = Arrays.copyOfRange (arr, m+1, r+1);
    int i=0, j=0, k=l;
      long swaps =0;
    while (i< left.length && j< right.length){
        if (left[i] <= right[j]){
            arr[k] = left[i];
              k++;  i++;
        }
      else {
          arr[k] = right[j];
              k++;  j++;
              swaps += (left.length - i);
        }
    }
    while (i< left.length){
          arr[k++] = left[i++];
      }
      while (j<right.length){
            arr[k++] = right[j++];
    }
    return swaps;
  }
}
```